시간차 학습 (Temporal-Difference Learning)

TD(0) 한 스텝 갱신 · SARSA와 CliffWalking · Q-learning과 경로 비교

Machine Learning 2 · 6주차

한국과학영재학교 (KSA)

Week 6

이번 주차 개요

- **6.1 TD(0)** — **TD 오차**, MC vs TD의 편향-분산 트레이드오프
- **6.2 SARSA** — V 대신 $Q(s,a)$가 필요한 이유, on-policy 구현
- **6.3 Q-learning** — $\max_{a'}$ 한 줄로 off-policy, CliffWalking 경로

이 주차의 두 갈림

"어떤 가치함수를 갱신하느냐"는 6.1→6.2($V \to Q$), "목표값의 다음 행동을 어떻게 채우느냐"는 6.2→6.3($a' \to \max$)에서 바뀐다. 이 두 갈림을 붙잡으면 DQN(Week 9)·액터-크리틱(Week 11)까지 이어지는 on-policy / off-policy의 큰 축이 미리 놓인다.

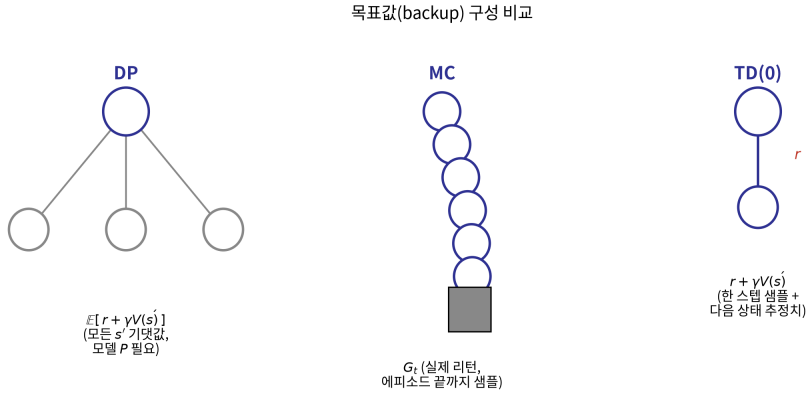
6.1 TD(0): 한 스텝만 보고 갱신한다, TD vs MC

위치: TD(0)는 DP와 MC의 정가운데

- **TD는 한 스텝만 진행해 즉시 갱신**
- **모델 불필요, 기다림 없는 절충안**

	목표값	대가
DP	기댓값 **정확히** 계산	모델 P, R 필요
MC	에피소드 끝의 **실제 리턴** 평균	끝까지 기다림
TD(0)	한 스텝 보상 + **다음 상태 추정치**	매 스텝 즉시

그림으로: DP·MC·TD의 목표값(backup) 구성



DP는 모든 다음 상태의 기댓값을, MC는 에피소드 끝까지의 실제 경로를, TD(0)는 단 한 스텝의 실제 경험 + 다음 상태의 **추정치**를 목표값으로 쓴다.

TD(0) 갱신식

$$V(s) \leftarrow V(s) + \alpha\big[r + \gamma V(s') - V(s)\big]$$

TD 오차(TD error)

대괄호 안 $r + \gamma V(s') - V(s)$. “아직 확실하지 않은 추정치를 가지고 스스로를 갱신한다”는 뜻에서 **부트스트래핑**(bootstrapping)이라 부른다.

- MC: 목표 = 실제 리턴 G_t(에피소드 끝까지의 실제 보상 합)
- TD: 목표 = $r + \gamma V(s')$(한 스텝 실제 보상 + 다음 상태 **추정치**)

7-state random walk — 환경과 참값

- 상태 $0 \sim 6$이 일렬. 매 스텝 확률 $1/2$로 좌/우로 1칸 이동.
- $0, 6$은 **터미널**. **매 스텝** 보상 $+1$(“아직 끝나지 않은 시간”이 가치에 반영되도록).

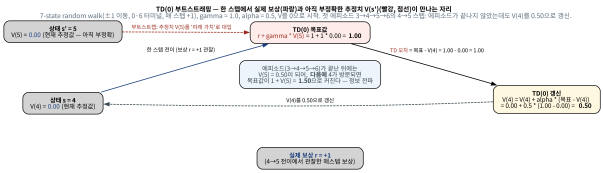
$V^*(i)$**는 기대 흡수 시간**

$$V^* = \big[0,\ 5,\ 8,\ 9,\ 8,\ 5,\ 0\big] \quad \big(T_i = i(6-i)\big)$$

손계산: 첫 에피소드 $3 \to 4 \to 5 \to 6$

$\alpha = 0.5$, 초기 $V = [0,0,0,0,0,0,0,0]$, 운 좋게 오른쪽 끝에 한 번에 닿는 에피소드. 실제 리턴 $G = +3$. 매 스텝 $r = 1, \gamma = 1$:

스텝	전이	목표 $r + \gamma V(s')$	갱신
t	$3 \to 4$	$1 + 0 = 1$	$V(3) \leftarrow \mathbf{0.5}$
$t{+}1$	$4 \to 5$	$1 + 0 = 1$	$V(4) \leftarrow \mathbf{0.5}$
$t{+}2$	$5 \to 6$(터미널)	$1 + 0 = 1$	$V(5) \leftarrow \mathbf{0.5}$



한 스텝 전이에서 실제 보상 r(파랑)과 부정확한 추정치 $V(s')$이 목표값 $r + \gamma V(s')$에서 만난다 — 에피소드가 끝나지 않았는데도 $V(4)$를 0.5로 갱신한다.

첫째 — 한 에피소드 만에 "방향"은 맞췄지만 "크기"는 틀림 (참값 $9 \cdot 8 \cdot 5$에 $0.5 \cdot 0.5 \cdot 0.5$).

MC라면 이 한 에피소드에서 $V(3) = V(4) = V(5) = G = 3$ 로 **한 번에** 갱신(매방문, $\alpha = 1$).
둘 다 참값과 멀지만, MC = "unbiased한 큰 점프", TD = "biased한 작은 점프".

둘째: 정보 전파와 셋째: 로컬성

- $V(5) = 0.5$**가 알려지자 V(4) 목표값 1.5로 자동 갱신**
- **Temporal-Difference의 본체: 한 스텝씩 시간차 전파**
- **갱신 목표는 에피소드와 무관**

로컬성: 부트스트래핑의 힘이자 약점

그 스텝에서 관찰된 r과 그 순간의 추정치 $V(5)$만으로 목표가 정해진다. 약점(편향)이 어떻게 “분산이 작음”으로 환원되는지 숫자로 확인한다.

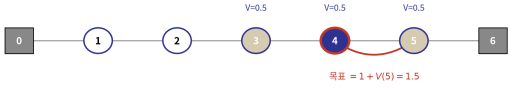
그림으로: 정보가 한 스텝씩 시간차로 전파된다

정보 전파: $V(5)$의 갱신이 $V(4)$의 목표값을, $V(4)$가 $V(3)$의 목표값을 바꾼다

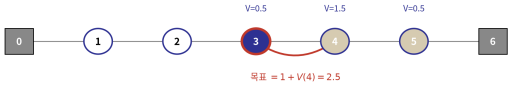
에피소드 1 종료: 3 → 4 → 5 → 6



에피소드 2: 상태 4를 다시 방문 → 5



에피소드 3: 상태 3을 다시 방문 → 4



TD vs MC: 편향과 분산

	몬테카를로	시간차 학습(TD)
목표값	실제 리턴 G_t(끝까지)	$r + \gamma V(s')$(한 스텝+추정치)
갱신 시점	에피소드 종료 후	매 스텝 즉시
편향/분산	편향 없음(불편), **분산 큼**	편향 있음(추정치 사용), **분산 작음**

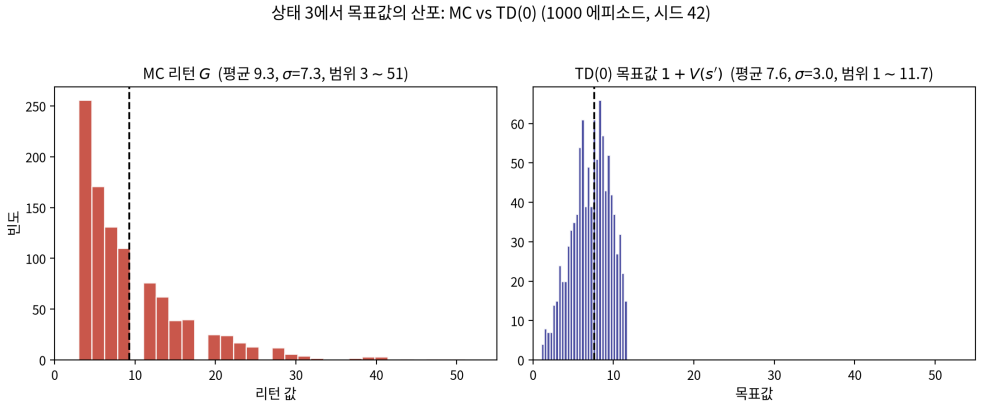
- **MC는 편향 0, 분산 큼**
- **TD는 편향 있음, 분산 작음**

목표값 분포를 숫자로: 1000 에피소드

방법	$V(3)$ 추정	쓴 목표값	목표값의 산포
MC(매방문)	≈ 9.3	리턴 G	평균 9.3, 표준편차 ≈ 7.3, **범위 3∼51**
TD(0), α=0.1	≈ 10.0	$1 + V(s')$	평균 7.6, 표준편차 ≈ 3.0, 범위 1∼11.7
참값 $V^*(3)$	9	—	—

- MC의 각 목표값(리턴)은 **에피소드 전체의 운**을 담는다 — 3스텝에 끝나기도, 51스텝을 방황하다 끝나기도.
- TD의 각 목표값은 “보상 1 + 옆 상태 추정치”뿐 — V가 자리를 잡으면 5∼10의 좁은 범위에서만 다님(표준편차는 MC의 절반 이하).

그림으로: 목표값 분포 — MC는 넓고, TD는 좁다



MC 리턴은 에피소드가 얼마나 방황했는지에 따라 3∼51까지 넓게 퍼지지만, TD(0) 목표값은 “보상 1 + 옆 상태 추정치”뿐이라 훨씬 좁게 모인다.

“TD의 분산이 작다”는 무엇이냐

- **분산: 갱신 목표값의 흔들림**

부트스트래핑의 가격표

$V(s')$가 부정확한 한, TD의 목표값은 참값을 **체계적으로 어긋나 있는**(= 편향된) 상태다. MC의 목표값 G는 어떤 에피소드가 아무리 운이 없어도 **평균**으로는 정확하다(불편). “작은 분산”을 얻는 대가로 “임시적인 편향”을 쓴다 — 그리고 그 편향은 V가 수렴할수록 **자동으로 사라진다**.

TD 오차: “예측 오차”로 읽는 갱신식

$$V(s) \leftarrow V(s) + \alpha \underbrace{\left[r + \gamma V(s') - V(s)\right]}_{\text{TD 오차 } \delta}$$

- 두 것의 차이가 예측 오차:

$$\delta = \underbrace{r + \gamma V(s')}_{\text{사후 평가}} - \underbrace{V(s)}_{\text{사전 예측}}$$

- $\delta > 0$: “예상보다 좋았다” $\rightarrow$ 추정치 **올림**. $\delta < 0$: “예상보다 나빴다” $\rightarrow$ **내림**.

$\delta = 0$인 지점: 벨만방정식

- $\delta = 0 \iff V(s) = r + \gamma V(s')$ (확률적 환경에선 기댓값으로) — 이 바로 **벨만 기대방정식**.
- 즉 “TD 오차가 계속 0이 되는 상태” = 벨만방정식의 해 = **참값** V^{π}.

이 신호는 책의 마지막까지

“예측 오차 신호”는 Week 11 액터-크리틱의 **크리틱의 오차** (어드밴티지)로 이어지는 TD 오차의 일반화다.

실습: 7-state random walk에서 TD(0)

노트북 핵심은 다음 함수 **하나**. 세 줄의 코드가 전부다:

```python
import random

def td0_random_walk(seed=42, n_episodes=1000, alpha=0.1, gamma=1.0, start=3):
    rng = random.Random(seed)
    V = [0.0] * 7                       # 터미널 V(0)=V(6)은=0 항상고정
    for _ in range(n_episodes):
        s = start
        while s not in (0, 6):
            sp = s + rng.choice([-1, 1])
            r = 1.0                     # 매스텝  +1
            V[s] += alpha * (r + gamma * (0.0 if sp in (0, 6) else V[sp]) - V[s])
            s = sp
    return V

V = td0_random_walk()
print([round(v, 2) for v in V])   # 참값 [0, 5, 8, 9, 8, 5, 에0] 가까움
```

코드 세 줄 읽기 — “모델이 없다”는 뜻

- (1) `sp = s + rng.choice([-1, 1])` — 환경의 유일한 무작위성.
- (2) `V[s] += alpha * (...)` — TD(0) 갱신식 자체.
- (3) `(0.0 if sp in (0, 6) else V[sp])` — **터미널 처리**.

“모델을 쓰지 않는다”는 것의 의미

우리가 $P(s' \mid s)$**를 알고 있느냐는 이 예제에서는 부차적.** 알고리즘이 갱신식에 쓰는 것은 **실제로 관찰된** (s, r, s') 한 줄뿐이다.

고정점: TD(0)는 어디로 수렴하나

TD(0) 고정점은 V^{π}

MC의 수렴 조건

- 에피소드가 **유한**히 끝나고
- 모든 상태가 충분히 방문

(불편한 샘플 평균이므로)

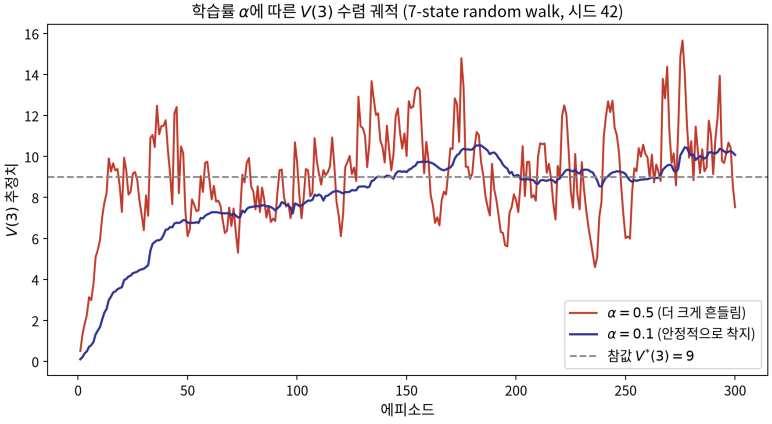
TD(0): MC보다 한 가지 까다로움

- 에피소드가 유한한 것
- 모든 상태가 **무한히** 방문
- 학습률 α가 **적절히 줄어**가는 것

Robbins-Monro 조건

$\sum_k \alpha_k = \infty$ 이고 $\sum_k \alpha_k^2 < \infty$. 이유(Week 2 증분 평균의 직관): α가 계속 크면(예: 항상 0.5) 잡음이 섞인 목표값이 계속 들어와, 추정치가 참값에 "착지"하지 못하고 **주위를 맴돈다**.
노트북에서 $\alpha = 0.5$ vs 0.1 비교 — 0.5 쪽이 참값 9 근처에서 더 크게 흔들린다.

그림으로: α가 크면 참값 근처에서 계속 흔들린다



자주 하는 실수 (1/2)

- 갱신을 먼저, 종료 판단을 나중에

자주 하는 실수 (2/2)

- **TD는 편향, MC는 불편**

확인 문제

1. (손계산) 7-state random walk, $\alpha = 0.5$, $V_0 = 0$. 첫 에피소드 $3 \to 4 \to 5 \to 6$. (a) 갱신 직후 $V(3), V(4), V(5)$는? (b) **다음** 에피소드에서 4가 다시 5로 이동할 때 목표값은 얼마로 바뀌는가? 이 차이가 정보 전파의 어떤 방향(5→4?)을 보여주는가? (c) 같은 에피소드를 MC(매방문, $\alpha = 1$)로 처리했으면?
2. (개념) "TD의 분산이 작다"에서 분산의 대상이 **목표값**임을 숫자(MC 리턴 범위 3∼51, $\sigma \approx 7.3$ vs TD 목표값 $\sigma \approx 3.0$)로 설명하라. "MC는 편향이 없다" 의 **정확한** 조건(무엇의 평균, 언제 성립)은?
3. (코드 진단) `break`가 갱신 **앞**에 있는 루프 (다음 프레임) — 어떤 상태의 학습을 빼먹게 되는가?

확인 문제 3: 코드 진단

루프의 문제를 찾아, 그것이 **어떤 상태의 학습**을 빼먹게 되는지 설명하라:

```
while s not in (0, 6):
    sp = s + rng.choice([-1, 1])
    r = 1.0
    if sp in (0, 6):
        break                              # ←갱신전후   / 어디에두었는가 ?
    V[s] += alpha * (r + gamma * V[sp] - V[s])
    s = sp
```

- **힌트**: break는 V[s] += ... **전**에 있다. $5 \to 6$ 스텝의 갱신은 어디에 갔는가?
- (4) (개념) $\gamma = 0$이면 갱신식은 $V(s) \leftarrow V(s) + \alpha(r - V(s))$ 로 줄어든다. $V(s')$이 목표값에 들어오지 않는다는 직관을 바탕으로, 이 환경의 참값 $V^*(i)$가 모든 내부 상태에서 몇이 되는지 구하고, “내다보기 0스텝”이라는 표현과 어떻게 일치하는지 설명하라.

역사: “Temporal-Difference” 이름의 유래

- **TD는 시간 방향의 차이**
- **TD는 한 스텝 앞 추정치 비교**

1983, mountain car

Barto · Sutton · Anderson(1983)의 *Neural learning of the optimal way to climb a mountain car*에서 처음 제안. **mountain car** 문제: 계곡 바닥의 자동차가 모멘텀을 만들어 언덕 중 어느 쪽이든 올라가게. 물리 모델이 복잡해 DP로 정확히 풀기 어렵고, MC처럼 에피소드를 기다리면 너무 느린 — **정확히 TD가 필요한** 환경.



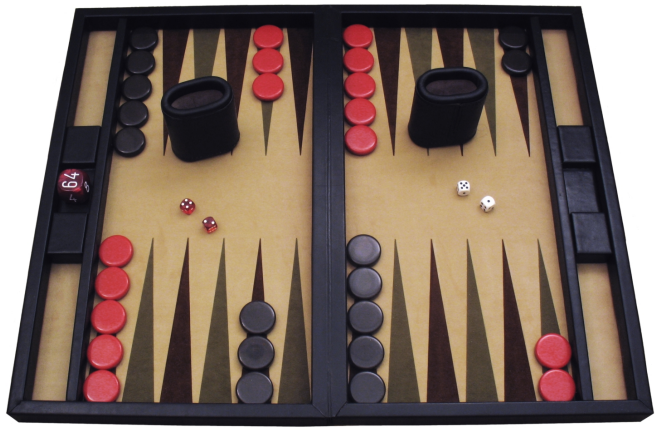
Andrew Barto(좌) & Richard Sutton(우) — 2024 ACM A.M. Turing Award (“강화학습의 개념적·알고리즘적 기초”)

자주 묻는 질문 (1/2)

- **Q. "TD(0)의 (0)"은?** 한 스텝만 앞을 내다보고 바로 부트스트래핑(0-step lookahead 이후 부트스트랩). Week 7의 n-step TD는 n 스텝 실제 관찰 후 부트스트랩: $n = 1$이면 TD(0), $n = \infty$(끝까지)면 사실상 MC.
- **Q. 부정확한 $V(s')$를 쓰는데도 왜 정확히 수렴하나?** 처음엔 $V(s')$이 부정확하나, 매 갱신마다 그 오차가 TD 오차만큼 줄고, 줄어든 값이 다시 다음 목표값에 쓰이면서 오차가 **전체적으로** 계속 줄어든다 — Week 4.3 바나흐 고정점과 같은 원리(차이: 샘플 기반).
- **Q. 에피소드 내 갱신 순서(in-place)가 결과에 영향?** 단기적으로는 조금, **수렴 결과(고정점)에는 영향 없음**. synchronous와 in-place는 둘 다 같은 고정점 V^π로 수렴 — **일관되게** 한 방식을 쓰면 된다.

자주 묻는 질문 (2/2)

- **Q. “Temporal difference”가 실제로 차이를 이루는 양은?**
  $\delta = r + \gamma V(s') - V(s)$ 안에서 $V(s') - V(s)$가 “$t{+}1$ 추정치와 t 추정치의 차”.
  관찰된 r에 더해 **인접한 두 시점의 가치 추정치의 차**가 학습 신호를 만든다.
  MC(리턴과 추정치의 차)와 DP(정확한 기댓값과 추정치의 차)는 이 “차”의 한쪽을 추정치가 아닌 것으로 채우므로 “temporal difference”라 부르지 않는다.
- **Q. 정책이 하나뿐이라 “예측”만 하고 “개선”은 안 하나?** 그렇다 — 이번 절(5.1절 MC 예측과 함께)은 **정책 예측** 문제. “더 나은 정책으로 갈아타기”는 6.2절에서 $V(s) \to Q(s,a)$로 바꾸고, 행동 선택이 $\arg\max_a Q(s,a)$로 읽히는 순간부터. TD 갱신식 뼈대는 **전혀** 바뀌지 않는다 — 바뀌는 것은 “어떤 가치함수를 갱신하느냐”뿐.



백가몬 보드 — TD-Gammon(Gerald Tesauro, IBM, 1992)이 **자가대국**만으로 TD(λ)를 학습해 세계 챔피언급 수준에 도달한 실제 게임

6.2 SARSA와 CliffWalking 실습

$V(s)$에서 $Q(s,a)$로

- **Q-함수 $Q(s,a)$ 학습**

벨만방정식과 같은 방식으로 정의 — 다음 상태를 **행동 a로** 이동한다는 것만 다름:

$$Q^{\pi}(s,a) = R(s,a) + \gamma \sum_{s'} P(s' \mid s,a)\, V^{\pi}(s')$$

V와 Q의 관계 — 한 줄

$V^{\pi}(s) = \sum_a \pi(a \mid s)\, Q^{\pi}(s,a)$: **상태 가치는 그 상태의 행동 가치들의 정책가중 평균**. Q를 알면 V는 이 평균으로 바로 얻어지지만, 반대(V에서 행동을 고르는 것)는 P가 필요 — **Q가 이득**.

탐욕 정책: Q표에서 바로 행동이 읽힌다

$$Q^{\pi}(s,a) = R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \sum_{a'} \pi(a' \mid s')\, Q^{\pi}(s',a')$$

- **탐욕 정책: Q표에서 행동 선택**
- Q를 학습한다 = "가치 평가"와 "행동 선택"을 **한 표로** 모두 해결.

작은 예 — Q표의 모양을 잡자

상태 셋 장난감 MDP: A(출발)$\cdot B \cdot G$(도착, 터미널). A에서 `right` $\to -1$로 B, `down` $\to$ 절벽 -100 받고 A로. B에서 `right` $\to G$(보상 0, 종료). $\gamma = 1$.

작은 예: backward induction

$$Q^*(B, \text{right}) = 0, \quad Q^*(A, \text{right}) = -1 + 1 \cdot 0 = -1$$
$$Q^*(A, \text{down}) = -100 + Q^*(A, \text{right}) = -101$$

- A의 두 행동: -1 대 -101 — **어떤 행동이 좋은지가 표의 한 행에 그대로 적혀 있다.**
- Q**는 행동 비교 결과를 미리 계산**

SARSA: 실제로 고른 행동을 쓴다

SARSA: 실제로 고른 행동 a'의 Q값을 목표로 TD 갱신

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma Q(s',a') - Q(s,a) \right]$$

- s, a — 현재 상태와 거기서 **행동 정책(behavior policy, 여기서는 ε-greedy)으로 고른** 행동.
- r, s' — 환경이 **관찰되게** 주는 보상과 다음 상태.
- a' — s'**에 도착한 뒤에** 다시 같은 행동 정책으로 고르는 행동.

SARSA 코드

```python
import random

def epsilon_greedy(Q, s, epsilon, n_actions):
    if random.random() < epsilon:
        return random.randrange(n_actions)
    return max(range(n_actions), key=lambda a: Q[s][a])

def sarsa_train(env_step, n_states, n_actions, n_episodes, alpha, gamma,
                epsilon, start_state):
    Q = [[0.0] * n_actions for _ in range(n_states)]
    for _ in range(n_episodes):
        s = start_state
        a = epsilon_greedy(Q, s, epsilon, n_actions)
        for _ in range(200):
            ns, r, done = env_step(s, a)
            na = epsilon_greedy(Q, ns, epsilon, n_actions)
            Q[s][a] += alpha * (r + gamma * Q[ns][na] - Q[s][a])  # 실제로고른  na
            s, a = ns, na
            if done:
                break
    return Q
```

SARSA는 지금 실제로 따르는 정책(**탐험을 포함한** ε**-greedy** 그 자체)의 가치를 학습한다 — **on-policy** (6.3절 Q-learning과의 핵심 차이가 바로 여기서).

무엇에 수렴하나: Q^b이지 Q^*가 아니다

SARSA 고정점은 행동 정책 b의 벨만 기대방정식

$$Q(s,a) = R(s,a) + \gamma \sum_{s'} P(s' \mid s,a) \sum_{a' \sim b} Q(s',a')$$

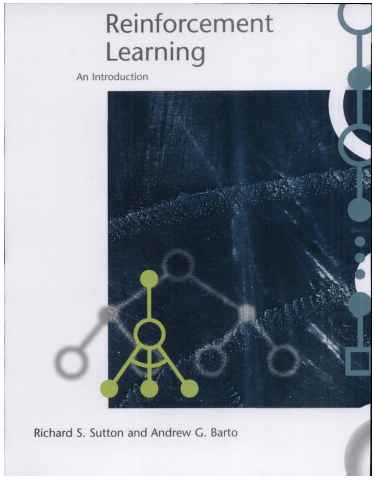
핵심

SARSA가 수렴하는 곳은 b **정책 자체의 Q-함수 Q^b이지, 최적의 Q^*가 아니다**. “내가 지금 이렇게(탐험까지 포함해) 행동할 때의 가치”를 배우는 것.

CliffWalking 환경

CliffWalking: 위험한 최단경로 vs 안전한 길

항목	내용
상태	48개 (0∼47), $(\text{row}, \text{col}) = \text{divmod}(s, 12)$
행동	4개: 0=위, 1=오른쪽, 2=아래, 3=왼쪽
출발/목표	$(3, 0)$ = 36 / $(3, 11)$ = 47 (터미널, 보상 0)
절벽	$(3, 1) \sim (3, 10)$ — 밟으면 -100 + $(3, 0)$ 회귀
벽 밖 이동	**그 자리에 남고** 보상 -1
시간 벌점	이동 자체에도 매 스텝 -1



Sutton & Barto, Reinforcement Learning: An Introduction (2nd ed., 2018) — CliffWalking 예제의 원 출처

두 후보 경로 — 그리고 절벽은 최선이 아니다

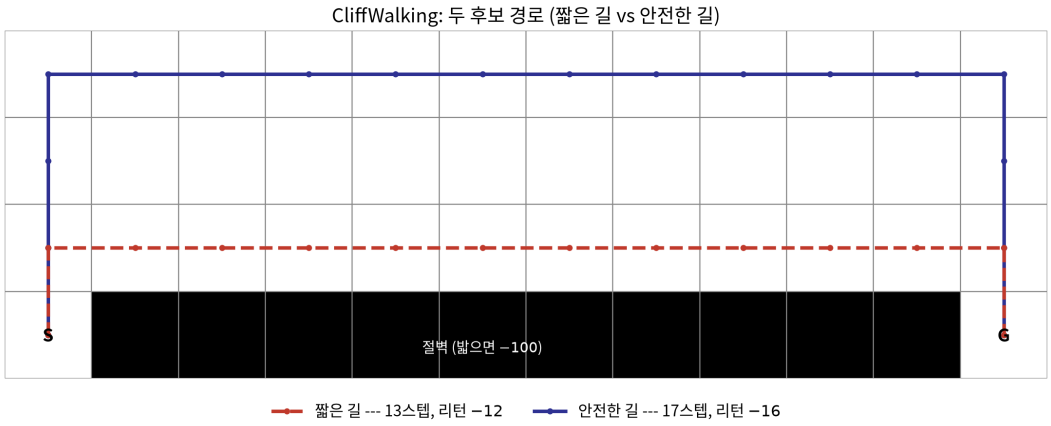
- **짧은 길: 13스텝, 리턴** -12
- **안전한 길: 17스텝, 리턴** -16

“절벽 자체가 최선”인 정책은 존재하지 않음

$\gamma = 1$ 최적 Q값에서 절벽 바로 위 $(2, 1)$의 Q(위,오른쪽,아래,왼쪽) = $-12,\ -10,\ -112,\ -12$ — down(절벽)은 압도적으로 나쁨. $\gamma = 0.99$로 바꾸면 $-11.4,\ -9.6,\ -111.3,\ -11.4$ — 수치는 달라져도 “절벽 행동의 Q가 ≈ -111로 벌어진다”는 구조는 동일.

절벽 옆 스치기 vs 멀리 돌아가기

그림으로: 두 후보 경로


CliffWalking: 두 후보 경로 (짧은 길 vs 안전한 길)
S
절벽 (밟으면 −100)
G
짧은 길 --- 13스텝, 리턴 −12
안전한 길 --- 17스텝, 리턴 −16

SARSA 실습 코드

```python
import gymnasium as gym

env = gym.make("CliffWalking-v1")
n_states, n_actions = env.observation_space.n, env.action_space.n

def sarsa_train_gym(env, n_episodes, alpha, gamma, epsilon):
    Q = [[0.0]*n_actions for _ in range(n_states)]
    for ep in range(n_episodes):
        s, _ = env.reset(seed=ep)
        a = epsilon_greedy(Q, s, epsilon, n_actions)
        for _ in range(500):
            ns, r, term, trunc, _ = env.step(a)
            na = epsilon_greedy(Q, ns, epsilon, n_actions)
            Q[s][a] += alpha*(r + gamma*Q[ns][na] - Q[s][a])
            s, a = ns, na
            if term or trunc:
                break
    return Q
```

결과 (500에피소드, $\alpha = 0.5, \gamma = 1.0, \varepsilon = 0.1$, 시드 42):

- 마지막 100에피소드 **평균 리턴** ≈ -31 수준.
- 학습 끝난 Q표로 **탐욕적**($\varepsilon = 0$) 롤아웃하면 **17스텝** — 절벽을 피하는 안전 경로.
- (6.3절에서 Q-learning과 나란히 비교)

배운 Q표: 경로를 읽어보자

1000에피소드 주 학습 결과. 두 칸만 떼어내도 스토리가 읽힌다:

상태	Q(위)	Q(오른쪽)	Q(아래)	Q(왼쪽)
$(3,0)$ 출발	-21.2	-124.7	-27.2	-35.5
$(2,1)$ 절벽 옆	-17.1	-59.0	-146.5	-20.1

- $(3,0)$에서 right $= -124.7$ — "여기서 절벽 방향으로 가겠다는 가치에 **추락 위험 전체**가 가격에 들어 있다" (ε-greedy 관점의 가치 평가).
- $(2,1)$에서 down $= -146.5$는 -100보다도 작음 — 추락 후 **출발점으로 돌아가 다시 걸어와야 하는** 대가까지 반영된 값.



칸 색 = $\max_a Q(s,a)$(어두울수록 낮음), 화살표 = 탐욕 행동. 파랑 실선 = SARSA가 배운 17스텝 안전 경로, 빨강 점선 = $\gamma = 1$ 최적 13스텝 경로. 검은 칸 = 절벽.

왜 안전 경로인가 — 메커니즘 1: 탐험 위험이 가격에

근사 계산(4행동, ε=0.1, γ=1, 추락 후 리턴 무시)	값
한 스텝에서 추락할 확률	$0.1 \times \frac{1}{4} = 0.025$
짧은 길 13스텝 중 절벽 바로 위 칸	10스텝 ($(2,1) \sim (2,10)$)
한 에피소드에서 최소 1번 추락할 확률	$1-(1-0.025)^{10} \approx 0.22$
짧은 길의 ε-greedy 기대 리턴	$\approx 13(-1) + 0.22(-100) \approx \mathbf{-35}$
안전한 길	$\approx 17(-1) \approx \mathbf{-17}$

같은 짧은 길을 걸어도, 탐험이 섞인 정책으로 걸으면 -35로, 안전 길로 걸으면 -17로 안는다.

왜 안전 경로인가 — 버그가 아니라 정석

핵심 판단

SARSA가 “탐험 포함 정책의 리턴”을 평가하는 알고리즘이므로, -17을 -35보다 **큰 가치**로 판단하고 안전하게 돌아가는 경로를 배우는 것은 **버그가 아니라 정석**.

- $(2,0)$에서 right의 목표값 $-1+\gamma\,\mathbb{E}_{a'\sim b}[Q(s',a')]$ 안에는 2.5%의 $Q(s',\text{down}) \approx -109$가 섞여 있다.

“절벽 옆을 따라 걷기”의 가치에 추락 위험이 **스스로 반영**되는 구조다. (위 메커니즘 1과 아래 손계산이 이 구조의 숫자.)

손으로 한 스텝: 무작위 down이 골랐을 때

$s = (2, 1)$(절벽 맨 왼쪽 칸 바로 위), $Q(s) = [\text{위} -1, \text{오른쪽} -1, \text{아래}\ 0, \text{왼쪽} -1]$. $\varepsilon = 0.1$-greedy가 **무작위로 down을 고름** → 절벽, $r = -100$, $s' = (3, 0)$ 회귀.
$Q(s') = [\text{위} -2, \text{오른쪽} -5, \text{아래} -9, \text{왼쪽} -4]$, s'에서 무작위 행동이 또 down이라 $a' = \text{down}$이 **실제로 골라짐**. $\alpha = 1$, $\gamma = 1$.

SARSA — 실제로 고른 a'의 값을 쓴다:

$$\text{목표} = -100 + (-9) = -109$$

$$Q(s, \text{down}) \leftarrow 0 + 1 \cdot (-109 - 0) = \mathbf{-109}$$

Q-learning(비교용) — 최선의 값을 쓴다:

$$\text{목표} = -100 + \max(-2, -5, -9, -4) = -102$$

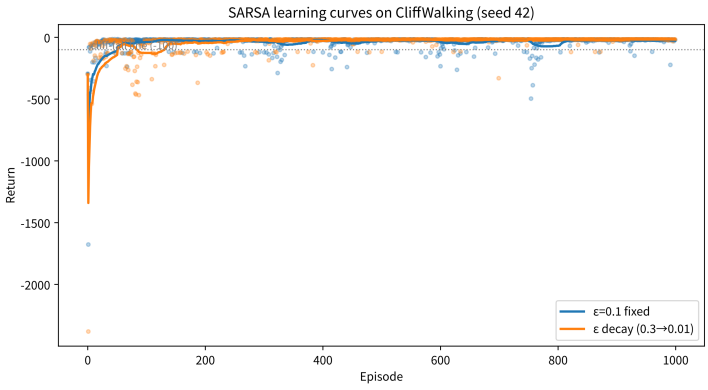
$$Q(s, \text{down}) \leftarrow \mathbf{-102}$$

본질

한 번의 갱신 차이는 7에 불과. 하지만 본질은 "7의 차이"가 아니라, **SARSA의 목표값에 "실제로 나쁜 행동을 고른" 사건 자체가 들어갔느냐**다.

학습 곡선: 추락이 사라지는 과정

- 첫 100에피소드 평균 리턴 약 $-65 \rightarrow$ 마지막 100에피소드 약 -19로 상승.
- 절벽 추락이 한 번 이상 일어난 에피소드 비율: **28%** $\rightarrow$ **0%**로 떨어짐.



이 학습 곡선은 6.3절의 Q-learning과 같은 환경에서 나란히 비교된다.

에피소드별 리턴(점)과 50에피소드 이동평균. 파랑 = $\varepsilon = 0.1$ 고정, 주황 = ε 감쇠 (0.3→0.01). 초반의 -100 추락이 명확히 보인다.

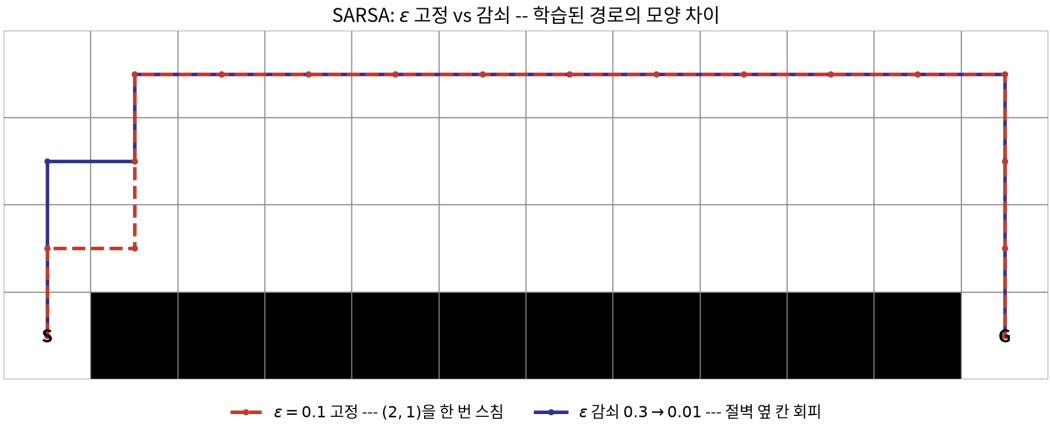
ε 감쇠: 경로의 모양까지 바뀐다

- 마지막 100에피소드 평균 리턴 ≈ -16.
- **최종 경로가 달라진다** —
  - $\varepsilon = 0.1$ 고정: 절벽 옆 칸 $(2,1)$을 **한 번** 스침.
  - decay: $(3,0) \to (2,0) \to (1,0) \to (1,1) \to (0,1) \to \cdots$ — 절벽 옆 칸을 **하나도** 스치지 않음.

왜

SARSA가 "탐험이 섞인 정책의 가치"를 배우는 알고리즘이므로, **학습 중의 ε이 어떤 값을 갖고 있었는가가 배운 경로의 모양을 결정**한다. (decay가 Q-learning에서 어떤 역할을 하는지는 6.3절에서 같은 환경으로 다시 본다.)

그림으로: ε 감쇠가 경로의 모양을 바꾼다



자주 하는 실수 (1/3)

- **SARSA 갱신 시 a'는 실제 고른 행동**

자주 하는 실수 (2/3): 학습 도중 리턴

- “배운 정책이 좋은가”를 판정하려면 $\varepsilon = 0$ **탐욕적 롤아웃**을 봐야 한다.

이 구분이 중요한 이유

“학습 도중 리턴(행동 정책)” vs “탐욕적 롤아웃(배운 정책)”의 구별은 6.3절의 “자주 하는 실수”에서 Q-learning과 나란히 다시 다뤄진다.

확인 문제

1. SARSA 갱신식 $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$의 다섯 요소 (s, a, r, s', a') 중 환경이 "관찰"로 주는 것과 에이전트가 "고르는" 것은 각각 무엇인가? 그리고 a'를 이동 **후** s'에서 고르는 것이 아니라, 이동 **전**에 고른 a를 그대로 a'로 써버리면 고정점이 더 이상 어떤 실제 정책의 벨만방정식도 아닌 이유를, "s'에서 정책이 실제로 고릴 행동 분포"와 비교해 설명하라.
2. (수치) $\varepsilon = 0.1$, 4개 행동, $\gamma = 1$. (a) 한 스텝에서 무작위 행동이 `down`일 확률은? (b) 짧은 길 13스텝 중 절벽 바로 위 10스텝을 지나며, 한 에피소드에서 **최소 1번** 절벽을 밟을 확률을 추정하라. (c) (b)를 써서 짧은 길의 ε-greedy 에이전트 한 에피소드 기대 리턴을 근사(추락 후 리턴 무시)하고, 안전 길(-17)과 비교해 "SARSA가 짧은 길을 피하는 이유"를 한 문장으로 완성하라.

확인 문제 3: 코드 진단

아래 SARSA 루프의 문제를 찾아, 그것이 **어떤 잘못된 학습 데이터**를 만들어내는지 설명하라:

```
for _ in range(500):
    ns, r, term, trunc, _ = env.step(a)
    na = epsilon_greedy(Q, ns, epsilon, n_actions)
    Q[s][a] += alpha*(r + gamma*Q[ns][na] - Q[s][a])
    s = ns            # ←무엇을빼먹었는가  ?
    if term or trunc:
        break
```

- **힌트**: 갱신식에는 na가 쓰이지만, 루프 상태는 `s = ns`만 갱신된다. 다음 `env.step(a)`에서 실행되는 `a`는 어느 상태의 행동인가?

역사: SARSA의 이름과 CliffWalking의 유래

- **SARSA = S-A-R-S-A**

1990년대 초: on/off-policy는 아직 중앙 개념이 아니었다

SARSA = “TD로 행동을 학습하는 **그냥** 방법”, Q-learning = “거기에 max을 집어넣은 **수정**”으로 제시. 그런데 “그냥 방법”과 “수정”이 **다른 고정점** (Q^b vs Q^*)에 수렴한다는 사실이 Sutton-Barto 교과서(1998)에서 **CliffWalking 예제**와 함께 정형화되면서, 비로소 강화학습을 지배하는 **근본 축**이 됨.

CliffWalking: on-policy vs off-policy 표준



Chris Watkins (Royal Holloway, Univ. of London) — 1989년 케임브리지 박사논문에서 Q-learning 제안

자주 묻는 질문 (1/2)

- **Q. `epsilon_greedy`는 Week 2.1의 그 함수와 완전히 같은 건가?** 그렇다 — 밴딧에서 "가장 좋아 보이는 팔을 $1-\varepsilon$로, 무작위 팔을 ε로 고른다"던 전략을, "팔" 대신 "행동", "추정 보상" 대신 "Q값"에 그대로 적용. 밴딧이 상태가 하나뿐인 특수한 MDP(Week 3.1 FAQ)라는 관점에서 이 재사용은 우연이 아니다.
- **Q. SARSA는 왜 매 스텝(온라인) 학습하나?** TD(0)의 정의 자체가 "한 스텝만 보고 즉시 갱신"(6.1절) — SARSA는 그 TD(0) 갱신을 $V(s)$ 대신 $Q(s,a)$에 적용한 것뿐. 같은 이유(끝날 때까지 기다릴 필요 없음)로 매 스텝 즉시.

자주 묻는 질문 (2/2)

- **Q. 안전한 경로를 배운 것이 "위험 회피(risk-averse)"인가?** 기술적으로는 **아니다** — SARSA에는 "위험 회피 효용함수" 같은 것이 없다. 다만 **탐험이 섞인 정책의 리턴을 평가**하는 알고리즘이므로, "절벽에 떨어질 확률"이 그 기대값의 한 항으로 **자동으로** 들어온다. 위험 회피로 **보이는** 것은 on-policy 성격의 **부수적 결과**. (진짜 "떨어질 확률 자체를 최소화" 목표가 필요하면 기대 리턴이 아닌 다른 목표(예: CVaR)를 설계해야 — 이 강의 범위 밖.)
- **Q. $\gamma = 1.0$ vs 0.99 어느 것을?** 에피소드 과업에선 둘 다 문제없다. $\gamma = 1$ = "보상이 언제 와도 동등하게 평가", 0.99 = "가까운 보상을 아주 조금 더 좋아한다". 이 실습은 6.3절 비교를 위해 1.0 — 0.99로 바꿔도 "절벽 회피"라는 질적 구조는 동일.

6.2 소결 — SARSA가 답하는 질문

"지금 실제로 이렇게(**탐험을 포함해**) 행동할 때, 이것이 얼마나 좋은가"를 평가 — 실제로 고른 행동을 TD 목표값으로 써 **행동 정책 자기 자신**의 질문에 답. 그 "답"의 모양 = 절벽을 피하는 **17스텝** 안전 경로, Q표 숫자(-124.7, -146.5)에 그대로 읽힘. 다음 6.3절은 이 갱신식의 한 항 $Q(s', a')$을 max으로 바꿔, 질문이 "**이상적인** 정책은 얼마나 좋은가"로 바뀔 때 경로를 본다.

6.3 Q-learning과 SARSA·Q-learning 경로 비교

위치: TD(0)는 두 갈래로 갈라진다

- 6.1: TD(0)는 “한 스텝만 보고, 다음 상태의 **추정치**로 부트스트랩”. 6.2: 그 갱신을 $Q(s, a)$에 적용한 **SARSA**.

갈림길

이 “실제로 고른 것”을 그대로 쓴다는 선택이 지금 6.3절의 갈림. **Q-learning**은 이 한 칸만 다르게 채운다 — 실제로 무엇을 골랐든 상관없이, **다음 상태에서 최선의 행동**을 가정하고 그 Q값을 목표로. 이 한 줄의 차이($Q(s', a')$ vs $\max_{a'} Q(s', a')$)가 두 알고리즘을 근본적으로 갈라놓으며, “실제 행동하는 정책의 가치”를 배우느냐 vs “이상적인 최적 정책의 가치”를 배우느냐의 문제로 이어진다.

Q-learning: 모델 없이, 최선의 다음 행동을 가정

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

```python
def q_learning_update(Q, s, a, r, s_next, alpha, gamma):
    max_next_q = max(Q[s_next].values()) if isinstance(Q[s_next], dict) \
                 else max(Q[s_next])
    td_error = r + gamma * max_next_q - Q[s][a]
    Q[s][a] += alpha * td_error
    return Q
```

Q-learning은 off-policy로 최적 정책 가치를 학습

왜 max인가 — 같은 경험, 다른 목표값

손으로 한 스텝: s에서 a를 하고 $r = -1$ 받아 s'로. s'에서 두 행동 a_1, a_2: 현재 $Q(s', a_1) = 0$, $Q(s', a_2) = 5$. ε-greedy가 **무작위로** a_1 (Q값 0인 "나쁜" 행동)을 고름. $\alpha = 0.5$, $\gamma = 1.0$, 갱신 중인 $Q(s, a) = 0$.

SARSA(실제로 고른 a_1 사용):

$$\text{목표} = -1 + 1.0 \times 0 = -1$$

$$Q(s, a) \leftarrow 0 + 0.5(-1 - 0) = -0.5$$

Q-learning(최선 max $= 5$ 사용):

$$\text{목표} = -1 + 1.0 \times 5 = +4$$

$$Q(s, a) \leftarrow 0 + 0.5(4 - 0) = +2.0$$

부호가 다르다

같은 경험에 대해 SARSA는 -0.5로, Q-learning은 $+2.0$로 갱신. 이 차이가 곧 두 알고리즘이 "배우는 것"의 차이 — 6.2절 CliffWalking에서 이 차이가 **경로의 모양**으로 드러난다.

max 뒤에 숨은 “낙관적 부트스트래핑”

- 즉 다음 상태의 “최선”이라는 **가정 하나로, 실제로 겪지 않은 경로의 가치**까지 지금 추정치에 반영.

이 “가정”은 **두 날을 가진 칼**이다:

- **장점 — 탐험을 가속**
- **단점 — 낙관론이 위험을 과소평가**

on-policy vs off-policy: 같은 데이터, 다른 대상

	행동 정책 b	목표 정책 π	갱신에 쓰는 다음 행동	이름
SARSA	ε-greedy	**같은** ε-greedy	실제로 고른 a'	**on-policy**
Q-learning	ε-greedy	**탐욕적** ($\varepsilon = 0$)	$\arg\max_{a'} Q(s', a')$	**off-policy**

핵심

행동 정책(b, 실제로 행동을 고르는 정책)과 **목표 정책**(π, 가치를 평가/학습하려는 정책)의 관계로 표현: $b = \pi$이면 on-policy, 다르면 off-policy. SARSA = "내가 지금 이렇게(탐험 포함) 행동할 때"의 가치를 배우고, Q-learning = "나는 항상 최선만 고를 텐데"라는 **이상적 정책**의 가치를 배운다.

왜 중요도 샘플링이 필요 없는가 — 5.3절 연결

- 그런데 Q-learning은 off-policy임에도 **중요도 샘플링을 쓰지 않는다**.

이유

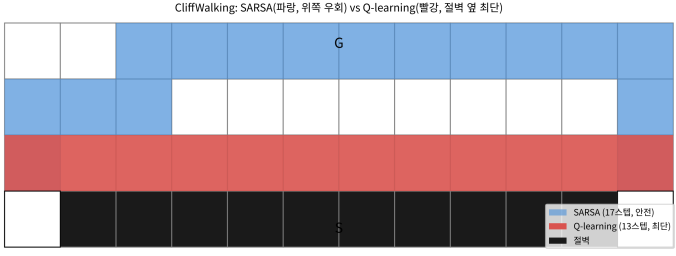
Q-learning은 "목표 정책의 **리턴 분포**를 샘플로 추정"하는 것이 아니라, **벨만 최적방정식의 고정점**을 샘플로 근사하기 때문. $\max_{a'} Q(s', a')$라는 목표값 자체가 "목표(탐욕적) 정책이었다면 다음에 이렇게 갔을 것"을 **직접** 부트스트랩 하므로, 행동 정책이 실제로 어떻게 움직였는지에 대한 보정(가중치)이 구조적으로 불필요.

TD는 한 스텝 목표값 사용으로 보정 불필요

실습: 두 알고리즘을 나란히

같은 CliffWalking에서 SARSA와 Q-learning을 각각 500에피소드 ($\alpha = 0.5, \gamma = 1.0, \varepsilon = 0.1$) 학습(스크래치패드 실행 검증):

학습 중(탐험 포함) 마지막 100에피소드 평균 리턴	
SARSA	-31.46
Q-learning	-53.33
학습 후 탐욕적($\varepsilon = 0$) 실행 리턴	
SARSA가 배운 정책	-17.0
Q-learning이 배운 정책	-13.0



두 숫자는 **각각 다른 방향**으로 읽어야 한다 — 아래 두 프레임에서.

Q-learning(빨강, 13스텝)은 절벽 바로 위(2행)를 스치는 짧은 길, SARSA(파랑, 17스텝)는 절벽에서 멀리 떨어진 위쪽(0∼1행)을 우회하는 안전 길. 절벽(검정)은 3행.

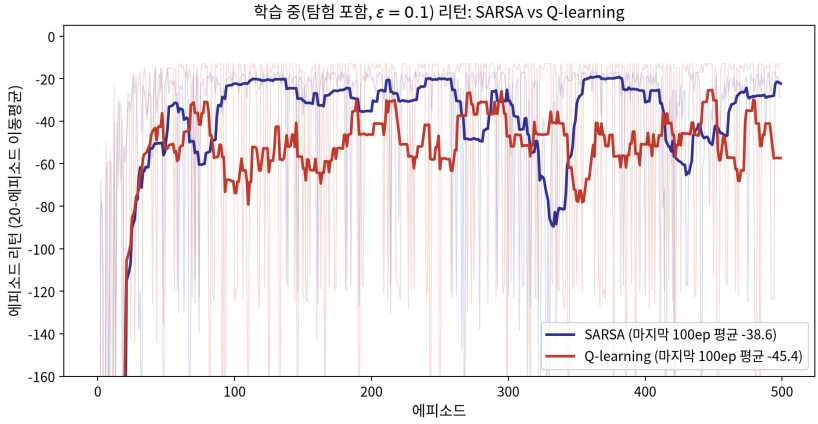
읽기 1: 학습 후 탐욕적 실행(-17 vs -13)

- **Q-learning 13스텝 vs SARSA 17스텝**
- **목표(탐욕적) 정책으로만 보면**, Q-learning의 13스텝이 **진짜 최단**.

읽기 2: 학습 중 평균 리턴(-31 vs -53)

- 학습 도중 SARSA가 더 좋은 성적

그림으로: 학습 도중 리턴 곡선 — SARSA가 더 유리

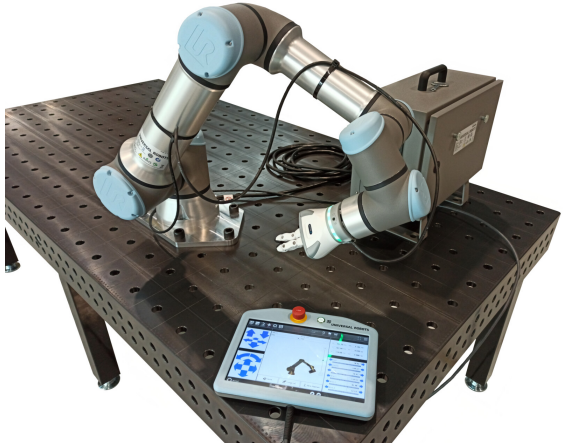


핵심: 답하는 질문이 다르다

Q-learning vs SARSA

Q-learning = “**이상적으로 항상 최선을 다한다면**”이라는 가정 아래 최적 경로를 찾고, SARSA = “**실제로 가끔 실수(탐험)할 수 있다**”는 현실을 감안한 경로를 찾는다. 어느 쪽이 “더 낫다”가 아니라, **애초에 답하는 질문이 다르다**.

- 실전 선택 기준:
  - “학습 **도중에도** 안전이 중요한가”(로봇 제어처럼 실제 사고가 나면 안 되는 경우) → **SARSA** 계열 선호.
  - “학습은 시뮬레이터에서 안전하게, 최종 **배포만** 최적이면 되는가” → **Q-learning**도 무방.



Universal Robots UR16e 협동로봇(cobot) 팔 — 학습 도중 실제 충돌·사고 위험이 있는 환경에서는 SARSA 계열이 유리

ε을 줄여가며: 탐험에서 착취로

학습 초반(ε 큼)

- 거의 모든 행동이 시도되어 Q값들이 충분히 방문.
- $\max_{a'} Q(s', a')$가 **믿을 만한** 값이 됨.
- 수렴 조건의 "모든 (상태,행동) 쌍이 무한히 방문"을 실질적으로 만족.

학습 후반($\varepsilon \to 0$)

- 행동이 거의 탐욕적.
- 학습 도중 리턴이 최종 성능을 반영.
- Q-learning은 여기서 "절벽 옆 최단 경로"를 **안정적으로** 학습.

decay 유무가 배운 경로 모양을 바꾼다

자주 하는 실수 (1/2)

- **Q-learning vs SARSA:** max**()의 유무**
- **터미널 상태에서** max**을 계산한다.** 목표(종료) 상태에서는 다음 상태가 없으므로 $\max_{a'} Q(s', a')$는 **0**이어야. `max(Q[goal])`를 0이 아닌 값으로 쓰면 "한 번 더" 갱신되어 **리턴에 실제로 존재하지 않는 미래 보상까지 더하는** 오차. 항상 `max_next_q = max(Q[ns]) if not done else 0.0` 처럼 `done` 플래그로 0을 강제.

자주 하는 실수 (2/2)

- **Q-learning과 SARSA는 다른 질문에 답한다**

확인 문제

1. SARSA 갱신식과 Q-learning 갱신식 중 **SARSA**에 해당하는 코드를 골라라: (a) `Q[s][a] += alpha(r + gammaQ[ns][na] - Q[s][a])` (b) `Q[s][a] += alpha(r + gammamax(Q[ns]) - Q[s][a])`. 고른 답에 대해 on-policy인지 off-policy인지, `na`(실제로 고른 다음 행동)가 목표값에 포함되는지를 근거로 설명하라.
2. 위 "손으로 한 스텝"에서 $Q(s', a_1) = 0$, $Q(s', a_2) = 5$ 대신 $Q(s', a_1) = Q(s', a_2) = 3$(두 행동 Q값이 같음)이었다면, SARSA와 Q-learning은 **같은** 목표값을 계산하는가? 같은 경험을 받았을 때 두 갱신식의 차이가 $Q(s', \cdot)$의 값 분포에 따라 어떻게 달라지는지 설명하라. (힌트: max가 "실제로 고른 것"과 언제 일치하고 언제 다른가?)
3. CliffWalking에서 $\varepsilon = 0$(탐험 없음)로 고정하고 학습시켰다면, SARSA와 Q-learning이 배운 경로는 **같은가?** 이유를, "행동 정책 = 목표 정책"인 경우 on/off-policy 차이가 소멸한다는 관점에서 설명하라. (힌트: $\varepsilon = 0$이면 "실제로 고른 a'"와 "arg max$_{a'}$ $Q(s', a')$"이 언제나 같은 행동.)

역사: 이름 붙이기가 차이를 요약한다

- **Q-learning:** $Q(s, a)$ **샘플 학습**
- **SARSA: 실제 행동 Q값 갱신**

이름 붙이기가 차이를 요약

Q-learning은 **무엇을** 배움(모델 없이 Q-함수)에 초점, SARSA는 **어떻게** 갱신(실제 행동을 따름)에 초점. 같은 TD(0) 뼈대 위에, “max을 넣느냐/넣지 않느냐” 한 줄이 **off-policy(이상적) / on-policy(현실)** — 강화학습을 지배하는 **가장 근본적인 축**을 만들어냄.

자주 묻는 질문 (1/2)

- **Q. 학습 후 ε을 0으로 낮추면 SARSA도 결국 짧은 길을 택하나? 아니다** — SARSA가 학습한 Q값 자체가 "탐험이 있는 상황"을 전제로 계산된 값이라, 그 Q값 기준으로 탐욕적으로 행동해도 **여전히 안전한(먼) 경로**를 선호하는 정책이 나온다. Q값을 다시 학습시키지 않는 한, ε을 실행 시점에만 낮춰도 저장된 Q값 자체는 바뀌지 않는다.
- **Q. 실전에서 하나를 골라야 한다면?** 먼저 "학습 자체가 실제 위험한 환경에서 일어나는가"(로봇이 실제로 넘어지거나 다칠 수 있는 상황)를 물어라 — 그렇다면 **SARSA**류가 유리. 반대로 시뮬레이터에서 안전하게 충분히 학습한 뒤 배포 시 **최적 성능만** 중요하다면, 목표 정책 자체의 최적성을 직접 학습하는 **Q-learning** 같은 off-policy 방법이 유리.

자주 묻는 질문 (2/2)

- **Q. Q-learning의 max은 "다음 상태의 모든 행동"에 대한 건데, 행동이 연속이거나 매우 많으면?** 이 강의의 **Table Q-러닝**(행동을 이산적으로 열거)에서는 max이 유한한 표에서 최대를 찾는 단순 연산. 행동 공간이 연속(로봇 각도·속도)이거나 매우 크면 표 자체가 불가능해져 **함수 근사**(Week 9 DQN)로 넘어가야 — 이때 $\max_a Q(s', a)$는 "함수 $Q(s', a)$를 행동 a로 관해 최대화"로 일반화. **원리는 같고**, "표에서 max"가 "최대화(optimization)"로 바뀜.
- **Q. 둘 다 수렴하나, 하나만?** **둘 다**, 같은 조건(모든 (상태,행동) 쌍이 무한히 방문 + α가 Robbins-Monro)이면. 다만 **수렴하는 값이 다르다**: Q-learning은 Q^*(최적 행동가치), SARSA는 ε-greedy 행동 정책 자체의 Q^π. "둘 다 수렴"이라 해도 **도달하는 지점이 다르다** — "다른 질문"이 수렴 결과값으로도 나타난다.

왜 Q-learning이 수렴하는가 (직관)

- Q-learning은 그 값을 직접 계산하지 않고 **샘플 기반 업데이트만으로** 근사하는데도, 정확히 그 값에 도달한다는 것이 증명됨:
  - 충분히 모든 상태-행동 쌍을 **무한히 방문**하고
  - 학습률 α를 **적절히 줄여**나가면

  참값 $Q^*(s,a)$로 수렴(Robbins-Monro).
- **TD 오차 0은 벨만 최적방정식 만족**

SARSA의 수렴 직감도 똑같이

다만 고정점이 Q^*가 아니라 ε**-greedy 정책의** Q^π. SARSA의 TD 오차 0인 지점은 $Q(s,a) = r + \gamma\, \mathbb{E}_{a' \sim b}[Q(s',a')]$ (행동 정책 b의 다음 행동에 대한 **기대**, max이 아님) = 그 행동 정책의 벨만 **기대**방정식.

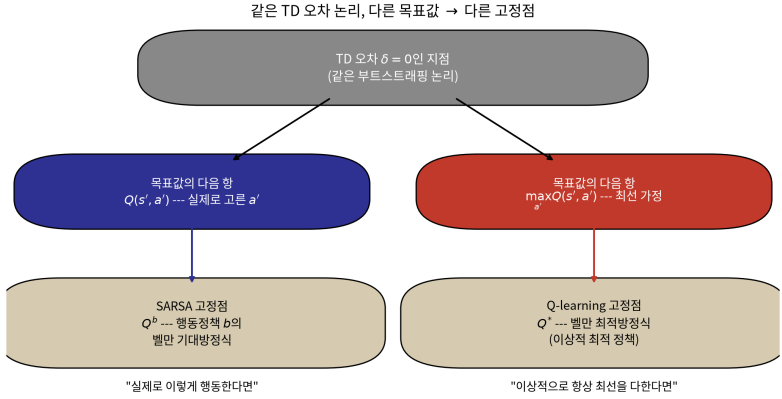
6.3 소결: 같은 부트스트래핑, 다른 고정점

- **TD 오차 0은 목표값에 따라 다른 고정점**

이 주차의 한 줄

시간차 학습은 MC의 “모델이 필요 없다”는 장점과 DP의 “한 스텝만 보고도 갱신한다”는 장점을 하나로 합친 것. Q-learning과 SARSA의 차이는, 똑같은 부트스트래핑 아이디어를 “**이상적인 목표 정책**”에 적용하느냐 “**실제 행동 정책**”에 적용하느냐 — **아주 작지만 결과는 크게 갈리는 선택**.

그림으로: 같은 TD 오차, 다른 고정점



이번 주차 정리

1. **TD(0)는 DP와 MC의 정가운데** — 모델 없이, 한 스텝 보상 + **다음 상태 추정치**로 매 스□□□ 즉시 갱신(부트스트래핑). MC(불편, 분산 큼) vs TD(편향, 분산 작음)의 트레이드오프는 "갱신 한 번의 목표값 흔들림"의 차이.
2. **SARSA = on-policy** — 실제로 **고른** 다음 행동 a'를 목표에 써, **행동 정책** b **자체의** Q^b를 학습. CliffWalking에서는 탐험 위험이 ε-greedy 가치에 가격으로 들어와 **절벽을 피하는 17스텝** 안전 경로를 배움.
3. **Q-learning = off-policy** — a'를 $\max_{a'}$로 바꾼 한 줄이 on/off-policy를 가르는 **전부**. "이상적인 최선만 고를 텐데"라는 낙관적 부트스트래핑으로 **절벽 옆 13스텝** 최단 경로를 배움. 학습 도중 리턴(-31 vs -53)은 **다른 질문**의 결과이지 품질이 아님.

다음 주 — Week 7. n-step TD와 적격흔적

TD(0)는 한 스텝만, MC는 끝까지 — "몇 스텝을 실제로 볼 것인가" 라는 하나의 다이얼 위의 양 끝. Week 7은 이 다이얼의 모든 지점을 아우르는 **n-step 부트스트래핑**과 **적격흔적**(eligibility traces)을 다루고, 경험뿐 아니라 **학습된 모델**까지 함께 쓰는 계획(planning, Dyna-Q)으로 나아간다.